

ชื่อกลุ่ม / เลขกลุ่ม

กลุ่ม 5

สมาชิกในกลุ่ม

ศุภณัฐ ถึงวงศ์

680112418040

x

Data Communication — Lab Worksheet

รู้จักเครื่องของตัวเองผ่าน SSH

แบบฝึกหัดภาคปฏิบัติ 4 ส่วน: พื้นฐาน Linux, การสื่อสารเครือข่าย, ความปลอดภัย และรูปแบบ client-server — ทำบน VM ส่วนตัวของนักศึกษาแต่ละคนบนเครื่อง bru-rdi

User	Host	Port
labman1	10.10.180.13	2201
labman2	10.10.180.13	2202
labman3	10.10.180.13	2203
labman4	10.10.180.13	2204
labman5	10.10.180.13	2205

เป้าหมายของ lab นี้

อ่านและอธิบายผลลัพธ์จากคำสั่งตรวจสอบระบบและเครือข่ายพื้นฐานของ Linux ได้

อธิบายเส้นทางของแพ็กเก็ตจาก VM ของตัวเองออกไปอินเทอร์เน็ต (private address → NAT → gateway)

เปิด/ปิดพอร์ตด้วย firewall (ufw) และอธิบายผลที่เกิดขึ้นกับการเชื่อมต่อ

สร้างและเรียกใช้บริการ client-server อย่างง่าย พร้อมโยกกับการ SSH เข้ามาเองว่าใช้หลักการเดียวกัน (port forwarding)

ส่วนที่ 1

พื้นฐานระบบ Linux

เริ่มจากรู้จักตัวตนของเครื่องที่เพิ่งต่อเข้ามา — ใครกำลังใช้งาน เครื่องชื่ออะไร รันระบบปฏิบัติการรุ่นไหน และมีพื้นที่เก็บข้อมูลเท่าไร

1.1

ตัวตนของผู้ใช้และเครื่อง

รันคำสั่งด้านล่าง เพื่อยืนยันว่ากำลังอยู่ใน VM ของตัวเอง ไม่ใช่เครื่องแม่ (host) แล้ววางผลลัพธ์ทั้งหมดที่ได้

```
labman?@vm1
```

```
$ whoami && hostname
```

```
copy
```

```
labman5
```

```
vm1
```

1.2

รู้ระบบปฏิบัติการและ kernel

เช็ค distro และเวอร์ชัน kernel ที่กำลังรันอยู่

```
labman?@vm1
```

```
$ cat /etc/os-release | head -3 && uname -r
```

```
copy
```

```
PRETTY_NAME="Ubuntu 22.04.5 LTS"
```

```
NAME="Ubuntu"
```

```
VERSION_ID="22.04"
```

```
5.15.0-190-generic
```

1.3

สิทธิ์ผู้ใช้และพื้นที่ดิสก์

ตรวจสอบ group ที่ตัวเองสังกัด และพื้นที่ดิสก์ที่เหลือใช้ไดบน root filesystem

```
labman?@vm1
```

```
$ id && df -h /
```

```
copy
```

```
uid=1000(labman5) gid=1000(labman5) groups=1000(labman5)
```

```
Filesystem      Size  Used Avail Use% Mounted on
```

```
/dev/vda1       15G  2.2G   13G  16% /
```

คำถาม: ทำไม id ถึงไม่แสดง group sudo แต่ตอนหลัง (ส่วนที่ 3) เราจะเรียก sudo ได้แบบไม่ถามรหัสผ่าน? เดาคะ

เหตุผลไว้ก่อน แล้วย้อนมาตรวจคำตอบทีหลัง

คำสั่ง id ไม่แสดงกลุ่ม sudo เพราะระบบไม่ได้ใส่ผู้ใช้ labman5 เข้าไปในกลุ่ม sudo แต่สาเหตุที่เรียกใช้งาน sudo ได้โดยไม่ถามรหัสผ่าน เป็นเพราะมีการเขียนกฎกำหนดสิทธิ์ NOPASSWD: ให้กับชื่อผู้ใช้ labman5 (หรือกลุ่ม labman5)

โดยตรงไว้ในไฟล์ /etc/sudoers หรือในไฟล์เตอร์ /etc/sudoers.d/

ส่วนที่ 2

การสื่อสารเครือข่าย

VM ของแต่ละคนอยู่หลัง NAT ส่วนตัว (คนละวงเครือข่ายภายใน แม้จะเห็นตัวเลข IP เดียวกัน) เราจะตามรอยแพ็กเก็ตตั้งแต่ interface ของตัวเอง ไปจนถึงปลายทางบนอินเทอร์เน็ต

2.1

Interface และ IP address ของตัวเอง

ดู network interface ที่มีอยู่ และ IPv4 address ที่ได้รับ

```
labman?@vm1
```

```
$ ip -4 addr show
```

```
copy
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
```

```
inet 127.0.0.1/8 scope host lo
```

```
valid_lft forever preferred_lft forever
```

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
```

```
inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
```

```
valid_lft 63138sec preferred_lft 63138sec
```

คำถาม: เทียบผลลัพธ์กับเพื่อนที่ใช้ labman คนอื่น — IP ที่ได้เหมือนกันหรือต่างกัน? ถ้าเหมือนกัน ทำไมจึงไม่ชนกัน?

IP ที่ได้จะเหมือนกัน (ได้ IP 10.0.2.15/24 เหมือนกันทุกคน)

สาเหตุที่ไม่ชนกัน: เพราะ VM แต่ละเครื่องแยกกันทำงานบน Virtual Network ส่วนตัวแบบ NAT (Network Address Translation) ของใครของมัน IP นี้จึงใช้สื่อสารแค่ภายใน VM เครื่องนั้นๆ เท่านั้น ไม่ได้อยู่บนเครือข่ายวงเดียวกันจริง

2.2

เส้นทางออกอินเทอร์เน็ต (default route)

ดู routing table เพื่อหา default gateway ที่แพ็กเก็ตทุกใบที่ไม่รู้ปลายทางจะถูกส่งไปก่อน

```
labman?@vm1
```

```
$ ip route
```

```
copy
```

```
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
```

```
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
```

```
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
```

```
10.0.2.3 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
```

คำถาม: gateway ที่เห็นคืออุปกรณ์ router จริงหรือไม่? คิดว่ามันคืออะไร

ไม่ใช่ Router จริง ครับ

gateway (10.0.2.2) ที่เห็นคือ Virtual Router / Virtual Gateway ที่จำลองขึ้นโดยซอฟต์แวร์ Virtual Machine (เช่น VirtualBox) ทำหน้าที่เป็นทางผ่านจำลองเพื่อทำ NAT (Network Address Translation) แปลงและส่งแพ็กเกจข้อมูลจากภายใน VM ออกสู่เครือข่ายภายนอกผ่านการเชื่อมต่อของเครื่อง Host จริง

2.3
ทดสอบการเชื่อมต่อด้วย ICMP

ทดสอบว่าปลายทางตอบกลับหรือไม่ ด้วยเลข IP ตรง ๆ ก่อน แล้วค่อยด้วยชื่อโดเมน

```
labman?@vm1
$ ping -c 3 8.8.8.8
$ ping -c 3 google.com
copy
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=7.23 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=8.44 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=7.19 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 7.188/7.617/8.436/0.579 ms
PING google.com (74.125.24.138) 56(84) bytes of data.
64 bytes from sf-in-f138.1e100.net (74.125.24.138): icmp_seq=1 ttl=255 time=30.1 ms
64 bytes from sf-in-f138.1e100.net (74.125.24.138): icmp_seq=2 ttl=255 time=30.0 ms
64 bytes from sf-in-f138.1e100.net (74.125.24.138): icmp_seq=3 ttl=255 time=30.1 ms
```

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 29.972/30.056/30.104/0.060 ms
คำถาม: คำสั่งที่สองต้องทำงานเพิ่มอีกหนึ่งขั้นตอนก่อนจะ ping ได้ ซึ่งคำสั่งแรกไม่ต้องทำ — ขั้นตอนนั้นคืออะไร และเกี่ยวข้องกับ protocol ชื่ออะไร?

ขั้นตอนที่ต้องทำเพิ่มคือ การแปลงชื่อโดเมน (Domain Name) ให้เป็นหมายเลข IP (Domain Name Resolution)

โดยเกี่ยวข้องกับโปรโตคอลชื่อ DNS (Domain Name System)

(คำสั่งแรก ping หา IP 8.8.8.8 ตรงๆ ได้ทันที ส่วนคำสั่งที่สองระบุเป็นชื่อ google.com ระบบจึงต้อง query ถาม DNS server เพื่อแปลงชื่อเป็น IP 74.125.24.138 ก่อนจึงจะเริ่ม ping ได้)

2.4
Domain Name Resolution
ก่อน query จริง ดูก่อนว่าเครื่องนี้ตั้งค่าให้ถาม DNS server ตัวไหนอยู่

```
labman?@vm1
$ resolvectl status
copy
Global
    Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
resolv.conf mode: stub
```

```
Link 2 (enp0s3)
    Current Scopes: DNS
        Protocols: +DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 10.0.2.3
DNS Servers: 10.0.2.3
```

คำถาม: DNS server ที่เห็น (บรรทัด Current DNS Server) คือ IP เดียวกับ default gateway ในข้อ 2.2 หรือไม่? ถ้าไม่ใช่ต่างกันอย่างไร และทำหน้าที่ต่างกันตรงไหน — แก่ผังเส้นทางแพ็กเก็ตของคุณจากข้อ 2.2 ให้เพิ่มจุด DNS server แยกออกมาให้ถูกต้อง

ไม่ใช่ IP เดียวกัน (DNS Server คือ 10.0.2.3 ส่วน Default Gateway คือ 10.0.2.2)

ความแตกต่างและการทำหน้าที่:

Default Gateway (10.0.2.2): ทำหน้าที่เป็น Virtual NAT Router คอยส่งผ่านแพ็กเก็ตข้อมูลเครือข่ายเข้า-ออกจาก VM ไปยังโลกภายนอก

DNS Server (10.0.2.3): ทำหน้าที่เป็น Virtual DNS Resolver คอยรับคำขอแปลงชื่อโดเมน (เช่น google.com) ให้เป็นหมายเลข IP

ตารางการจัดเส้นทาง (Routing Table) นี้แสดงว่า Default Gateway ของระบบคือ 10.0.2.2 ผ่านอินเทอร์เฟซ enp0s3 ซึ่งเป็น IP ของ Virtual Router/Gateway ในระบบ NAT ของ VirtualBox ที่ทำหน้าที่รับทราฟฟิกทั้งหมดที่จะออกไปยังเครือข่ายภายนอกหรืออินเทอร์เน็ต

[Non-Root User: labman5]

▼ (1) เรียกใช้คำสั่ง privileged เช่น 'sudo ufw status' หรือ 'sudo su'

/usr/bin/sudo

(2) อ่านไฟล์กำหนดสิทธิ์: /etc/sudoers (และ /etc/sudoers.d/*)

พบกฎ: labman5 ALL=(ALL:ALL) NOPASSWD: ALL

▼ (3) ข้ามขั้นตอนการขอรหัสผ่าน (NOPASSWD)

และยกระดับสิทธิ์ทันที

[Root Privilege]

(4) รันคำสั่งระดับ System Administration (UID 0)

- แก้ไขไฟล์คอนฟิกระบบ (/etc/)
- จัดการ Firewall (UFW / iptables)
- จัดการ Network Interfaces & Services

2.4b

ทดสอบ DNS query จริง

ใช้ dig เพื่อดูขั้นตอน DNS resolution ที่เกิดขึ้นเบื้องหลังก่อนคำสั่ง ping ในข้อ 2.3 จะทำงานได้

ลองเอง

\$ dig google.com

copy

; <<>> DiG 9.18.39-0ubuntu0.22.04.6-Ubuntu <<>> google.com

;; global options: +cmd

```
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41092
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1
```

```
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;google.com.                IN      A
```

```
;; ANSWER SECTION:
google.com.      184     IN      A       74.125.24.102
google.com.      184     IN      A       74.125.24.100
google.com.      184     IN      A       74.125.24.139
google.com.      184     IN      A       74.125.24.138
google.com.      184     IN      A       74.125.24.113
google.com.      184     IN      A       74.125.24.101
```

```
;; Query time: 4 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Sep 14 07:09:00 UTC 2026
;; MSG SIZE rcvd: 135
```

คำถาม: หา section ANSWER — IP ที่ได้ตรงกับที่ ping google.com ไว้ในข้อ 2.3 หรือไม่? บันทึก Query time เทียบกับ RTT ของ ping แล้วดูว่า SERVER ที่ dig ติดต่อ (ท้ายผลลัพธ์) ตรงกับ DNS server ที่เห็นในข้อ 2.4 หรือไม่
IP Address: IP Address ที่ได้ใน section ANSWER (กลุ่ม IP 74.125.24.xxx) ตรงกับ IP ที่คำสั่ง ping google.com ในข้อ 2.3 เลือกใช้ (โดยจะตรงกับ IP ตัวใดตัวหนึ่งใน 6 รายการนี้ที่ DNS คืนค่ามา)

Query time vs RTT: Query time ของ dig เท่ากับ 4 msec ซึ่งมีค่าน้อยกว่าเวลา Round Trip Time (RTT) รวมในการส่ง แพ็กเกจของคำสั่ง ping อย่างมาก เนื่องจาก dig ทำหน้าที่แค่สอบถามข้อมูลชื่อโดเมนจาก DNS server เท่านั้น โดยไม่ได้รอรับ-ส่งข้อมูลแพ็กเกจจริงไปกลับถึงเครื่องปลายทางของ Google เหมือน ping

SERVER: DNS Server ที่ dig ติดต่อคือ 127.0.0.53#53 ซึ่งเป็น local stub resolver (systemd-resolved) บน Ubuntu โดยจะ ตรงกับ DNS Server ที่ปรากฏในข้อ 2.4

2.5 Socket ที่กำลังเปิดรอรับการเชื่อมต่อ

ดูว่ามี service อะไรกำลัง listen บนพอร์ตไหนบ้างในเครื่องนี้

```
labman?@vm1
```

```
$ ss -tuln
```

```
copy
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
udp	UNCONN	0	0	127.0.0.53%lo:53	0.0.0.0:*	
udp	UNCONN	0	0	10.0.2.15%enp0s3:68	0.0.0.0:*	
tcp	LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*	
tcp	LISTEN	0	128	:::22	:::*	

คำถาม: พอร์ต 22 และพอร์ต 53 ที่เห็น เกี่ยวข้องกับสิ่งที่คุณกำลังทำอยู่ตอนนี้อย่างไรบ้าง?

พอร์ต 22 (TCP - OpenSSH Server): เป็นพอร์ตบริการ SSH ที่ใช้สำหรับเปิดรับการเชื่อมต่อให้คุณ (หรือเครื่องอื่น) สามารถリモートเข้ามารันคำสั่งและควบคุมเครื่อง Linux เครื่องนี้ผ่าน Terminal/Command line ได้

พอร์ต 53 (UDP/TCP - systemd-resolved DNS): เป็นพอร์ตบริการ DNS Resolver (ผูกอยู่กับ IP 127.0.0.53) ที่ทำหน้าที่แปลงชื่อโดเมนให้เป็น IP Address ซึ่งคอยให้บริการเบื้องหลังกับคำสั่งที่คุณรันไปก่อนหน้านี้ เช่น ping google.com และ dig google.com

สรุปส่วนที่ 2

วาดเส้นทางแพ็กเก็ตจาก VM → gateway → อินเทอร์เน็ต ลงกระดาษ พร้อมกำกับ IP ทุกจุด

อธิบายได้ว่าทำไม VM ทุกตัวมี IP ภายในซ้ำกันแต่ไม่ชนกัน

ส่วนที่ 3

ความปลอดภัยเบื้องต้น

การเชื่อมต่อของคุณเข้ามาที่ VM นี้ ไขกลไกความปลอดภัยอยู่แล้วโดยไม่รู้ตัว — มาดูกันว่ามันทำงานอย่างไร แล้วลองเปิด/ปิด firewall เอง

3.1

บริการที่คุณกำลังใช้อยู่ตอนนี้

ตรวจสอบสถานะของ SSH daemon ที่กำลังให้บริการ session ปัจจุบันอยู่

```
labman?@vm1
```

```
$ systemctl status ssh --no-pager
```

```
copy
```

- ssh.service - OpenBSD Secure Shell server

Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)

Active: active (running) since Mon 2026-09-14 06:20:45 UTC; 59min ago

Docs: man:sshd(8)

man:sshd_config(5)

Main PID: 8013 (sshd)

Tasks: 1 (limit: 2307)

Memory: 4.0M

CPU: 143ms

CGroup: /system.slice/ssh.service

└─8013 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

คำถาม: คุณเชื่อมต่อเข้ามาได้โดยไม่ใส่รหัสผ่านเลย — อธิบายกลไกที่ทำให้เป็นแบบนั้น (คำใบ้: ดูที่ ~/.ssh/)

การเชื่อมต่อโดยไม่ต้องใส่รหัสผ่านเกิดขึ้นเพราะใช้กลไก SSH Key-based Authentication โดยเครื่องเซิร์ฟเวอร์นำ Public Key ไปบันทึกไว้ในไฟล์ ~/.ssh/authorized_keys เมื่อเราส่ง SSH เข้ามา ระบบจะตรวจสอบการจับคู่กับ Private Key บนเครื่องต้นทางโดยอัตโนมัติ หากคีย์ตรงกันจึงอนุญาตให้เข้าใช้งานได้ทันทีโดยไม่ต้องป้อนรหัสผ่าน

3.2

สิทธิ์ผู้ดูแลระบบ (sudo)

เรียกสิทธิ์ root ผ่าน sudo — สังเกตว่ามีการถามรหัสผ่านหรือไม่

```
labman?@vm1
```

```
$ sudo whoami
```

```
copy
```

```
root
```

คำถาม: VM นี้ตั้งค่าไม่ให้อ่านรหัสตอน sudo เพื่อความสะดวก — ถ้าเป็นเครื่อง production จริง ความเสี่ยงของการตั้งค่าแบบนี้คืออะไร และควรแก้ด้วยวิธีไหน?

ความเสี่ยง: หากผู้บุกรุกเข้าถึงบัญชีผู้ใช้ธรรมดาได้ จะสามารถยกระดับสิทธิ์เป็น root เพื่อยึดและทำลายระบบทั้งหมดได้ทันทีโดยไม่มีปราการป้องกัน รวมถึงเสี่ยงต่อการที่ผู้ใช้ตั้งใจหรือเผลอรันคำสั่งอันตรายโดยไม่มีระบบเตือนสติ

วิธีแก้ไข: แก้ไขไฟล์ /etc/sudoers โดยลบตัวเลือก NOPASSWD: ออก เพื่อบังคับให้ผู้ใช้ต้องป้อนรหัสผ่านยืนยันตัวตน
ทุกครั้งเมื่อเรียกใช้งาน sudo

3.3
เปิดใช้งาน firewall ของตัวเอง

เปิด ufw พร้อมอนุญาตเฉพาะพอร์ต SSH ก่อนเปิดโหมด deny-by-default — ลำดับสำคัญมาก ถ้าเปิด enable ก่อน allow
อาจล็อกตัวเองออกจาก session ปัจจุบัน

ลองเอง

```
$ sudo ufw allow OpenSSH  
$ sudo ufw enable  
$ sudo ufw status verbose
```

```
copy  
Rules updated  
Rules updated (v6)  
Aborted  
Status: inactive
```

คำถาม: เปิด terminal อีกแท็บ (session ใหม่) — ยังต่อได้ปกติไหม? แล้วลอง sudo ufw deny 22/tcp จากนั้นเปิด session
ใหม่อีกครั้ง เกิดอะไรขึ้น บันทึกสิ่งที่สังเกตได้ (session เดิมที่เปิดค้างจะยังไม่หลุดทันที เพราะเป็น connection ที่
established ไปแล้ว — ทำให้เป็นแบบนั้น)

เปิด terminal อีกแท็บ (ก่อน deny): เชื่อมต่อได้ตามปกติ

หลังสั่ง sudo ufw deny 22/tcp แล้วเปิด session ใหม่: 'ไม่สามารถเชื่อมต่อได้ (Connection refused หรือ Connection timed
out) เนื่องจาก UFW Block ทราฟฟิกขาเข้าพอร์ต 22 สำหรับการเชื่อมต่อใหม่ทั้งหมด

3.4
ตรวจสอบการเข้ารหัสของ HTTPS

ดู response header ของเว็บที่ใช้ TLS เพื่อยืนยันว่าการเชื่อมต่อถูกเข้ารหัส

```
labman?@vm1  
$ curl -sI https://www.google.com  
copy  
HTTP/2 200  
content-type: text/html; charset=ISO-8859-1  
content-security-policy-report-only: object-src 'none';base-uri 'self';script-src 'nonce-MP6ufaiF20d_KaVDT7D5Zg' 'strict-dynamic'  
'report-sample' 'unsafe-eval' 'unsafe-inline' https: http;;report-uri https://csp.withgoogle.com/csp/gws/other-hp  
accept-ch: Sec-CH-Prefers-Color-Scheme  
p3p: CP="This is not a P3P policy! See g.co/p3phelp for more info."  
date: Mon, 14 Sep 2026 07:27:30 GMT  
server: gws  
x-xss-protection: 0  
x-frame-options: SAMEORIGIN  
expires: Mon, 14 Sep 2026 07:27:30 GMT  
cache-control: private  
set-cookie: __Secure-STRP=ANmZwa2erm_IjSzO4tohIva-  
o1YQN7QobNpzF8DeSh3novxq0EDNfRgJgcnMNdvXSKMLbi0W3O2ILuvWKXltB6u7IwXWlxN2M6GB; expires=Mon, 14-Sep-2026  
07:32:30 GMT; path=/; domain=.google.com; Secure; SameSite=strict  
set-cookie: AEC=AdJVEav9DWerQmG4D7SPsDdZI5SzcLJCIkXGCr7bvOnCnbp4QHroAp1AF9Q; expires=Sat, 13-Mar-2027  
07:27:30 GMT; path=/; domain=.google.com; Secure; HttpOnly; SameSite=lax  
set-cookie:  
NID=534=cG1rYY33yoSp17DHEQdnBRaSemcxL5VHtHFz06zpYNVe0EubbHWL8p1LsUtCsAycdJjmEFDcoKNCgTmu9MYikbV84xYV  
08Z5n-6gqytyL1LTGJq5bQWepupGJHgfKY4BH5nSiViuVCdngrqFLpAJCmFH-
```

```
44Opmk_Qx6Z3kFTIRZm1nneWn3uIFo7ymKGclDkHzfYgyWF538tgXwSTxUtGI9mjHV3; expires=Tue, 16-Mar-2027 07:27:30 GMT; path=/; domain=.google.com; HttpOnly  
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
```

คำถาม: เทียบกับ SSH ที่คุณใช้เชื่อมต่ออยู่ตอนนี้ ทั้งคู่เข้ารหัสข้อมูล แตต่างกันอย่างไรในแง่ protocol และวิธี handshake?

1. ด้าน Protocol:

HTTPS (HTTP/2) แยกส่วนการรับส่งข้อมูลออกจากระบบความปลอดภัย โดยทำงานซ้อนอยู่บนโปรโตคอล TLS/SSL อีกที ในขณะที่ SSH เป็นโปรโตคอลเบ็ดเสร็จในตัวเองที่รวมการรับส่งข้อมูลและการเข้ารหัสไว้ในสแต็กเดียว

2. ด้านวิธี Handshake:

HTTPS (TLS): ยืนยันตัวตนเซิร์ฟเวอร์ผ่าน Digital Certificate ที่ออกโดยองค์กรกลาง (CA)

SSH: ยืนยันตัวตนโดยตรงระหว่างสองเครื่องผ่าน Host Key (การจำและเชื่อถือคีย์ในการเชื่อมต่อครั้งแรก) และใช้ SSH Key-pair (Public/Private Key) ในการพิสูจน์ตัวตนผู้ใช้งาน

ส่วนที่ 4

Client-Server และ Port Forwarding

ปิดท้ายด้วยการสร้าง server ของตัวเอง แล้วย้อนกลับมาดูว่าการที่คุณ SSH เข้ามาได้ตั้งแต่ต้น lab นี้ ก็ใช้หลักการ "แปลงพอร์ต" แบบเดียวกัน

4.1

ตั้ง HTTP server ของตัวเอง

เปิด web server ง่าย ๆ ด้วย Python built-in module แล้วเรียกดูจากเครื่องเดียวกัน

```
labman?@vm1
```

```
$ echo "lab: hello from $(whoami)" > index.html
```

```
$ python3 -m http.server 8000 &
```

```
$ curl -s localhost:8000
```

```
copy
```

```
10.133.0.245 - - [14/Sep/2026 07:35:09] "GET / HTTP/1.1" 200 -
```

```
10.133.0.245 - - [14/Sep/2026 07:35:09] code 404, message File not found
```

```
10.133.0.245 - - [14/Sep/2026 07:35:09] "GET /favicon.ico HTTP/1.1" 404 -
```

```
182.232.75.67 - - [14/Sep/2026 07:36:10] "GET / HTTP/1.1" 200 -
```

```
182.232.75.67 - - [14/Sep/2026 07:36:11] code 404, message File not found
```

```
182.232.75.67 - - [14/Sep/2026 07:36:11] "GET /favicon.ico HTTP/1.1" 404 -
```

```
182.232.75.67 - - [14/Sep/2026 07:36:17] "GET / HTTP/1.1" 304 -
```

```
182.232.75.67 - - [14/Sep/2026 07:39:19] "GET / HTTP/1.1" 304 -
```

```
127.0.0.1 - - [14/Sep/2026 08:02:08] "GET / HTTP/1.1" 200 -
```

```
lab: hello from labman5
```

คำถาม: ในผลลัพธ์ฝั่ง server (access log) มีข้อมูลอะไรบ้าง (method, path, protocol, status code)? ระบุแต่ละส่วนจากบรรทัด log ที่ได้

ระบุส่วนประกอบจากบรรทัดตัวอย่าง 10.133.0.245 - - [14/Sep/2026 07:35:09] "GET / HTTP/1.1" 200 - ดังนี้:

HTTP Method: GET

Path: / (หรือ /favicon.ico)

Protocol: HTTP/1.1

Status Code: 200 (สำเร็จ), 404 (ไม่พบไฟล์), หรือ 304 (ดึงจากแคช)

4.1b

เปิดให้เข้าถึงจากภายนอกจริง

ระหว่างที่ `python3 -m http.server 8000` ในข้อ 4.1 ยังรันอยู่ เครื่องแม่ (bru-rdi) ได้ forward พอร์ต 8000 ของ VM คุณ ออกสู่อินเทอร์เน็ตแล้ว ที่พอร์ต 300 ต่อท้ายด้วยเลขท้าย username ของคุณ (เช่น labman3 → 3003, labman5 → 3005) เปิด `http://202.29.14.232:300X/` (แทน X ด้วยเลขของคุณ) จากอุปกรณ์ที่ไม่ได้อยู่ในวง network เดียวกับห้องเรียน (เช่น สลับมือถือไปใช้ 4G/5G)

คำถาม: เข้าถึงได้จริงหรือไม่? ระบุ URL เต็มที่ใช้ทดสอบ และอธิบายว่าทำไม IP ที่ใช้ตอนนี้ (202.29.14.232) ถึงต่างจาก IP ที่ใช้ตอน SSH ตั้งแต่ต้น lab (10.10.180.13) ทั้งที่เป็นเครื่องเดียวกัน

10.10.180.13 (Private IP): เป็น IP สำหรับใช้งานภายในเครือข่ายท้องถิ่น (Local Network) ของห้องเรียน/สถาบัน ทำให้สามารถใช้งานได้เฉพาะอุปกรณ์ที่เชื่อมต่อในวง LAN เดียวกันเท่านั้น

202.29.14.232 (Public IP): เป็น IP สาธารณะของเครื่องแม่ (bru-rdi) ที่ใช้ออกสู่อินเทอร์เน็ตภายนอก เมื่อมีการตั้งค่า Port Forwarding บนเครื่องแม่ (แมปพอร์ตภายนอก 3005 เข้ากับพอร์ต internal 8000 ของ VM คุณ) ทำให้อุปกรณ์จากเครือข่ายภายนอก เช่น 4G/5G สามารถวิ่งผ่าน Public IP เข้ามาถึง Web Server บน VM ของคุณได้

4.2

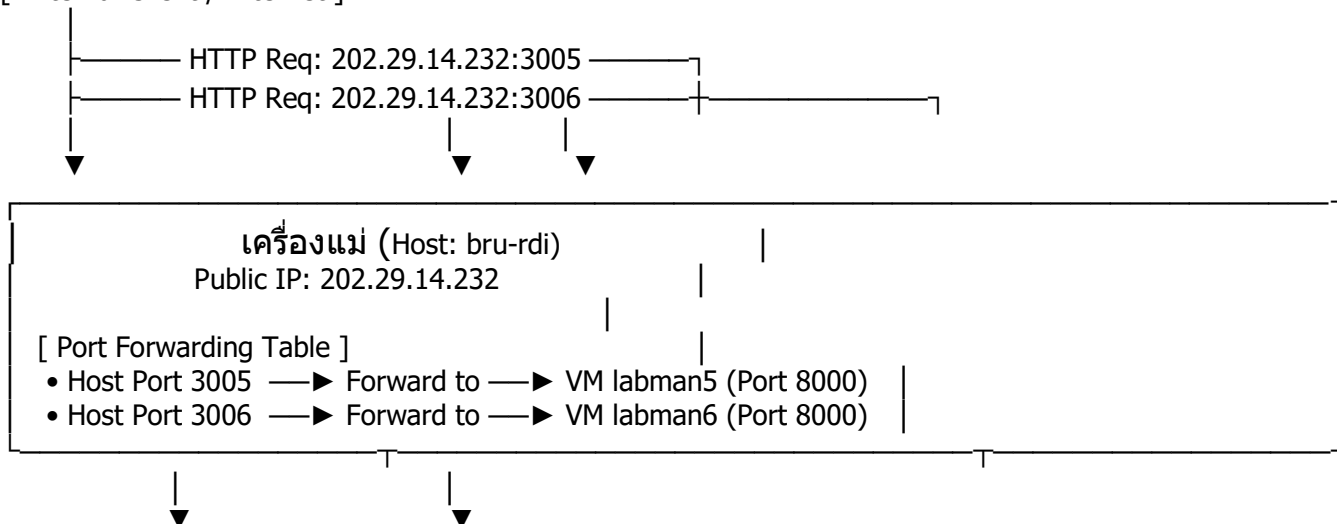
ย้อนดูว่าตัวเองเข้ามาได้ยังไง

เทียบสิ่งที่เพิ่งทำในข้อ 4.1-4.1b กับวิธีที่คุณเชื่อมต่อเข้ามาตั้งแต่ต้น lab

คำถาม: ตอนนี้เครื่องแม่มี port forwarding คู่ขนานกันอยู่ 2 คู่ (host 220X → guest 22 สำหรับ SSH, host 300X → guest 8000 สำหรับ HTTP) อธิบายว่าทั้งสองคู่ใช้กลไกเดียวกันอย่างไร และถ้ามี labman คนที่ 6 เข้ามาอยากเปิด web server ของตัวเองบ้าง เครื่องแม่ต้องเตรียมอะไรเพิ่ม (คำใบ้: พอร์ต 8000 ฝั่ง guest ของทุกคนซ้ำกันได้ไหม เพราะอะไร แล้วพอร์ตฝั่ง host ละ ซ้ำกันได้หรือไม่) วาดแผนภาพประกอบคำตอบ

1. หลักการทำงาน: ทั้งคู่ใช้กลไก Port Forwarding (DNAT) เหมือนกัน โดยเครื่องแม่จะแปลงแพ็กเก็ตที่วิ่งเข้ามายัง IP สาธารณะและพอร์ตที่กำหนด (Host Port) ให้ส่งต่อไปยัง IP ภายใน (Guest IP) และพอร์ตของบริการนั้นๆ (Guest Port) บน VM2. สิ่งที่เครื่องแม่ต้องเตรียมเพิ่มสำหรับ labman6: พอร์ตฝั่ง Guest (ซ้ำกันได้): พอร์ต 8000 (หรือ 22) บน VM แต่ละเครื่องซ้ำกันได้ เพราะแต่ละ VM แยกวงเครือข่ายจำลองและมี IP ภายในเป็นของตัวเอง พอร์ตฝั่ง Host (ห้ามซ้ำกัน): พอร์ตบนเครื่องแม่ห้ามซ้ำกันเด็ดขาด เพราะมี Public IP เดียว เครื่องแม่จึงต้องเพิ่ม Port Forwarding Rules ใหม่ คือ: SSH: Forward พอร์ต 2206 (Host) → ไปยังพอร์ต 22 (VM labman6) HTTP: Forward พอร์ต 3006 (Host) → ไปยังพอร์ต 8000 (VM labman6)

[External Client / Internet]



VM: labman5 • Internal IP: 10.0.2.15 • Guest Port: 8000 (Web Server)	VM: labman6 • Internal IP: 10.0.2.16 • Guest Port: 8000 (Web Server)
--	--

ส่งงาน

Diagram เส้นทางแพ็กเก็ต (ส่วนที่ 2)

คำตอบข้อ 3.2 และ 3.3 (firewall + sudo)

Screenshot การเข้าเว็บจากมือถือ (4G/5G) ในข้อ 4.1b

Diagram ตารางแปลพอร์ตข้อ 4.2 (SSH + HTTP คู่ขนานกัน)

Screenshot ผลลัพธ์คำสั่ง dig google.com จากข้อ 2.4b

Data Communication — VM Lab Worksheet

bru-rdi · qemu:///session · SLIRP NAT per VM

หน้าตางนี้ยังดาวน์โหลดไม่ได้

ดาวน์โหลดไฟล์แล้วส่งผ่านช่องทางส่งงานของวิชา (เช่น Google Classroom / LMS / อีเมล)

ดาวน์โหลดไฟล์คำตอบ